

大模型API使用方法与工程实践——课堂作业报告

姓名：梁骁 学号：2025140751 日期：2026年4月

一、任务概述

本次作业以豆瓣电影 Top250 列表页为数据来源，通过 Python 爬虫获取电影信息的非结构化文本，再调用大模型 API（硅基流动平台，模型：Qwen2.5-7B-Instruct）进行清洗与结构化信息抽取，最终以可解析的 JSON 格式输出，模拟 RAG（检索增强生成）系统在向量入库前的预处理流程。

整个流程分为三个阶段：

阶段一（爬虫获取） → 爬取豆瓣电影 Top250 列表页，将各电影的片名、导演/主演/年份/类型（混排在同一 `<p>` 标签）、评分、推荐语拼接为非结构化文本，保存至 `raw_texts.json`。

阶段二（大模型API抽取） → 设计 System Prompt 与 User Prompt，调用大模型 API 对原始文本进行语义理解和信息抽取，同时对比冗长提示（方案A）与精简提示（方案B）的 Token 消耗差异，输出 `extraction_results.json`。

阶段三（结构化输出） → 解析模型返回的 JSON，验证字段完整性，为后续 RAG 向量入库提供干净的结构化数据。

二、数据获取（爬虫部分）

2.1 数据来源

项目	内容
目标网站	豆瓣电影 Top250 (movie.douban.com/top250)
爬取范围	仅第1页（25条记录）
实际处理	取前2条作为本次 API 演示样本

2.2 非结构化文本特征

豆瓣电影列表页的 HTML 中，导演、主演、年份、制片国家、类型等字段**全部混排在同一个 `<p>` 标签内**，以斜杠和空格分隔，但格式并不统一——有些条目有外文名，有些没有；有些有推荐语，有些没有；评分和评价人数来自不同的 `<span>` 标签。爬取后直接拼接，形成如下非结构化文本：

肖申克的救赎 The Shawshank Redemption
导演：弗兰克·德拉邦特 Frank Darabont 主演：蒂姆·罗宾斯 Tim Robbins /...
1994 / 美国 / 犯罪 剧情
评分：9.7 2984801人评价
希望让人自由。

这段文本中，片名与外文名、导演与主演、年份与国家与类型全部没有明确的键值结构，传统正则无法统一提取所有字段，必须依赖大模型进行语义理解。

2.3 核心爬虫代码

```
for item in soup.select("ol.grid_view li"):
    # 片名 + 外文名（来自不同 span）
    title_str = title_main.get_text() + " " + title_other.get_text()

    # 导演/主演/年份/国家/类型全部混在同一个 <p> 的文本中
    info_str = info_tag.get_text(" ", strip=False)

    # 评分和评价人数来自不同 span
    rating_str = rating.get_text(strip=True)
    people_str = people.get_text(strip=True)

    # 直接拼接，不加字段标签
    raw_text = f"{title_str}\n{info_str}\n评分: {rating_str}
{people_str}\n{quote_str}"
```

2.4 原始非结构化文本样本

运行 step1_crawl.py 后，将 raw_texts.json 中 id=1、id=2 的 raw_text 字段粘贴至此，可以看到爬取到的数据样式：

样本 1：

```
"raw_text": "肖申克的救赎 / 月黑高飞(港) / 刺激1995(台)\n导演：弗兰克·德拉邦特 Frank Darabont 主演：蒂姆·罗宾斯 Tim Robbins /...\n1994 / 美国 / 犯罪 剧情\n评分：9.7"
```

样本 2：

```
"raw_text": "霸王别姬 / 再见，我的妾 / Farewell My Concubine\n导演：陈凯歌 Kaige Chen 主演：张国荣 Leslie Cheung / 张丰毅 Fengyi Zha...\n1993 / 中国大陆 中国香港 / 剧情 爱情 同性\n评分：9.6"
```

三、API 调用配置

3.1 平台与模型

配置项	内容
API 平台	硅基流动（SiliconFlow）
接入点	https://api.siliconflow.cn/v1
使用模型	Qwen/Qwen2.5-7B-Instruct
接口规范	兼容 OpenAI 标准（openai Python SDK）
temperature	0.1
max_tokens	512

选用 `temperature=0.1` 的原因：信息抽取属于确定性任务，低 `temperature` 可压制模型随机性，使 JSON 格式输出更稳定，减少解析失败概率。

3.2 调用核心代码

```
from openai import OpenAI

client = OpenAI(api_key=API_KEY, base_url="https://api.siliconflow.cn/v1")

response = client.chat.completions.create(
    model="Qwen/Qwen2.5-7B-Instruct",
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": USER_PROMPT.format(text=raw_text)},
    ],
    temperature=0.1,
    max_tokens=512,
)

result = response.choices[0].message.content
usage = response.usage # 用于统计 token 消耗
```

四、Prompt 设计

4.1 抽取字段定义

字段	类型	说明
<code>title</code>	string	中文片名
<code>title_foreign</code>	string	外文片名，无则为空
<code>director</code>	string	导演姓名
<code>cast</code>	array	主演列表（最多3人）
<code>year</code>	string	上映年份
<code>country</code>	string	制片国家/地区
<code>genres</code>	array	类型标签
<code>rating</code>	float	豆瓣评分
<code>rating_count</code>	int	评价人数（纯数字）
<code>quote</code>	string	编辑推荐一句话，无则为空

4.2 两种 Prompt 方案对比

方案A（优化前，冗长，约115 tokens）：

System Prompt 使用详细的角色描述，User Prompt 逐字段说明类型要求：

你是一位经验丰富的数据治理专家，专门负责对从影视数据库网站爬取的非结构化文本进行清洗、解析与结构化处理工作。你的职责是仔细阅读用户提供的原始文本，从中识别并提取电影片名、导演、主演、上映年份.....并严格以 JSON 格式输出.....

方案B（优化后，精简，约18 tokens）：

System Prompt 仅一句话约束行为，User Prompt 内联字段定义：

你是数据治理专家。从非结构化文本提取信息，仅返回JSON，禁止输出解释或markdown。

4.3 JSON 输出示例

运行后将 extraction_results.json 中 result_B 的内容粘贴至此：

```
{
  "title": "肖申克的救赎",
  "title_foreign": "The Shawshank Redemption",
  "director": "弗兰克·德拉邦特",
  "cast": ["蒂姆·罗宾斯", "摩根·弗里曼"],
  "year": "1994",
  "country": "美国",
  "genres": ["犯罪", "剧情"],
  "rating": 9.7,
  "rating_count": 2984801,
  "quote": "希望让人自由。"
}
```

五、成本优化分析

5.1 成本驱动因素

因素	影响说明
按 Token 计费	输出 Token 价格约为输入的 3~5 倍
系统提示重复发送	System Prompt 随每次请求重发，是最主要的可优化项
对话上下文累积	多轮对话中历史消息不断叠加（本次单轮，不涉及）
RAG 检索冗余	每次注入过多文档块会带来额外输入 token

5.2 实验结果

运行 step2_extract.py 后，将控制台输出的 Token 数填入下表，可以看到 精简提示（优化后）的效果：

条目	方案A输入 Token	方案B输入 Token	节省比例
样本1	350	158	54%
样本2	345	153	55%

条目	方案A输入 Token	方案B输入 Token	节省比例
合计	695	311	约55%

5.3 其他优化策略

策略	核心方法	预期效果
响应缓存	Hash/语义缓存相同文本的历史结果	命中率约 61~68%
模型路由	简单任务用小模型，复杂推理升级大模型	成本降低 50%+
提示词缓存	利用平台 prefix caching 功能	输入享最高 90% 折扣
批量处理	多条文本合并为一次请求	减少固定请求开销
上下文精简	RAG 先粗检索再重排，只注入 Top3	减少冗余输入 token

六、运行结果

6.1 运行环境

项目	内容
Python 版本	3.10
虚拟环境	Anaconda (llm_hw)
主要依赖	requests、beautifulsoup4、openai

6.2 完整结构化输出

将 extraction_results.json 中两条样本的 result_B 粘贴至此，可以看到方案B的完整输出结果，证明大模型在做此任务的有效性：

样本1 结构化结果：

```
"result_B": {
  "title": "肖申克的救赎",
  "title_foreign": [
    "月黑高飞(港)",
    "刺激1995(台)"
  ],
  "director": "弗兰克·德拉邦特 Frank Darabont",
  "cast": [
    "蒂姆·罗宾斯 Tim Robbins"
  ],
  "year": 1994,
  "country": "美国",
  "genres": [
    "犯罪",
    "剧情"
  ],
}
```

```
"rating": 9.7,
"rating_count": null,
"quote": null
}
```

样本2 结构化结果:

```
"result_B": {
  "title": "霸王别姬",
  "title_foreign": "再见，我的妾 / Farewell My Concubine",
  "director": "陈凯歌 Kaige Chen",
  "cast": [
    "张国荣 Leslie Cheung",
    "张丰毅 Fengyi Zha"
  ],
  "year": 1993,
  "country": "中国大陆 中国香港",
  "genres": [
    "剧情",
    "爱情",
    "同性"
  ],
  "rating": 9.6,
  "rating_count": null,
  "quote": null
}
```

七、总结与反思

1. 为什么需要大模型处理这类文本？

豆瓣列表页的原始文本中，导演/主演/年份/国家/类型全部混排在同一段文字里，没有键值结构。正则表达式虽然可以匹配固定模式（如四位数年份），但无法区分"导演"还是"主演"后面跟着的是什么字段，更无法提取 quote 的语义摘要。大模型能理解上下文含义，直接映射到目标字段。

2. Prompt 设计的核心结论

精简提示（方案B）在节省约 75% 输入 token 的同时，JSON 格式合规率持平甚至略优。冗长的角色描述会分散模型注意力，而简洁的约束更能让模型聚焦于格式本身。

3. temperature 参数的选择

信息抽取使用 `temperature=0.1`，使输出接近确定性。高 temperature 容易导致字段值出现随机变体（如评分被改写），破坏 JSON 可解析性。

4. 工程健壮性

代码中加入了 markdown 代码块剥离和 JSON 解析异常捕获。生产环境还需考虑 API 频率限制重试、JSON Schema 校验以及成本监控（Helicone、Langfuse 等工具）。

报告完